

ENGR 102 Summer Bridge

Day 13 Student Workbook

Loop Design, Break, Continue, and Nested Tracing

Wednesday, July 22, 2026 • 50 points

Name		UIN		Section	
------	--	-----	--	---------	--

Boolean-operator convention. Python operators appear as **and**, **or**, and **not**. Their lowercase spelling is required in code.

Daily target

increasing independence

Select skip, stop, and reset behavior while tracing repeated work inside repeated work.

Allowed tools	Prior tools plus break, continue, and nested for-loop tracing
Support supplied	Behavioral requirements and a sample input stream are supplied.
You must decide	Loop form, placement of skip and stop decisions, state updates, and reset scope.
Scaffold level	Requirements only; students select the control-flow pattern.

Scoring and completion standard

50 points

Evidence	Points	Secure work shows
Integrated trace	10	intermediate state, condition results, and exact output
Diagnosis and repair	8	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	32	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace**10 points**

Trace nested repetition while distinguishing a skipped pass, an early stop, and a per-station reset.

```
overall = 0

for station in range(1, 3):
    station_total = 0
    for reading in [0, station + 1, 5]:
        if reading == 0:
            continue
        if reading == 5:
            break
        station_total = station_total + reading
    overall = overall + station_total
    print(station, station_total)

print(overall)
```

Your task

1. Give a complete reached-pass ledger that identifies the continue, break, station_total reset, and overall update.
2. Write every output line and explain why the inner break does not end the outer loop.

A final answer without the requested state evidence cannot earn full trace credit.

Task 2. Diagnose and repair**8 points**

The intent is to print every odd integer through 5 and then stop. The current order omits 5.

```
value = 0
while value < 8:
    value = value + 1
    if value % 2 == 0:
        continue
    if value == 5:
        break
    print(value)
```

Your task

1. Give actual versus intended output, identify the ordering cause, and write a minimal repair that still stops at 5.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

Task 3. Construct a complete program**32 points across Pages 4-5****Program specification**

- Repeatedly read an integer inspection code.
- Code 0 ends normal input. A negative code is ignored and processing continues.
- A code of 90 or greater is a shutdown code: stop immediately without counting or adding it.
- Each positive code below 90 is accepted. Count accepted codes and add their values.
- After the loop, print `accepted_count` and total on separate lines.

Program workspace – begin the connected solution here.

Task 3 continued. Test and defend the program**included in 32 points**

Continue code if needed.

Required tests, expected results, and brief defense

--